Hydro

Lettuce hydroponic system: how it works, what has been built, and how to run it

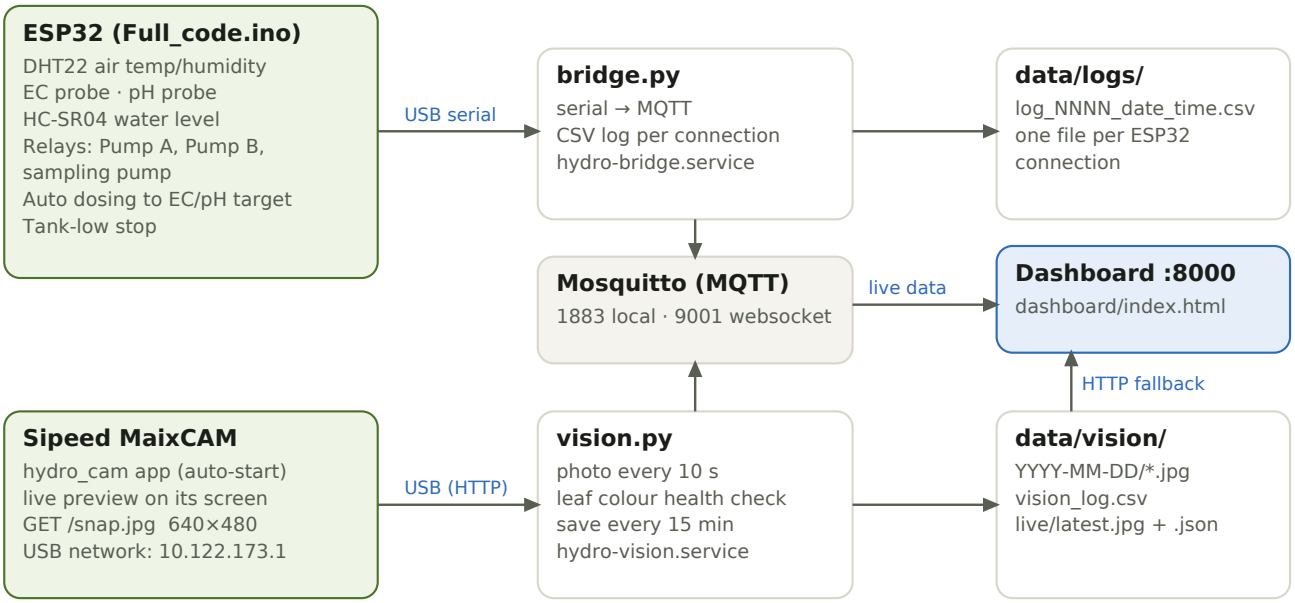
Raspberry Pi 5 · ESP32 · Sipeed MaixCAM · MQTT · Python

Status as of 24 September 2026 · Repository: github.com/muaz8172/hydro (private)

Contents	
1. System overview	What each part does and how data flows
2. Hardware and wiring	Pins, dividers, relays, camera link
3. How it works	Sensors, pump control, logging, camera, dashboard, image dataset
4. What has been done	Progress, commits and current status
5. How to run the system	First-time setup, daily use, commands
6. Calibration	EC, pH and tank level
7. Troubleshooting	Common problems and fixes
8. Known issues and next steps	Safety items and roadmap

1. System overview

The system grows lettuce in a hydroponic tank and watches it all day without internet. An **ESP32** reads the water and air sensors and switches the pumps. A **Raspberry Pi 5** collects everything, stores it, and serves a dashboard. A **Sipeed MaixCAM** camera photographs the plants so their health can be checked from leaf colour.



Everything talks through **MQTT** (the Mosquitto broker on the Pi). Each program publishes on its own topic, and the dashboard listens to all of them live.

MQTT topic	Sent by	Contents
`hydro/sensors`	bridge.py	One JSON reading every 2 s from the ESP32, plus `log_id` and `row` that match the CSV line
`hydro/log`	bridge.py	Text messages from the ESP32 (calibration, pump ON/OFF, tank low)
`hydro/cmd`	you (mosquitto_pub)	Commands forwarded to the ESP32 (e.g. `status`, `ph7`, `pumpauto`)
`hydro/vision`	vision.py	Plant health result (JSON) every 10 s
`hydro/vision/image`	vision.py	The latest camera photo (JPEG)
`hydro/vision/status`	vision.py	`online` or `offline: reason`
`hydro/vision/cmd`	dashboard	`snap` = save a photo to disk now

2. Hardware and wiring

Part	Pins	ESP32 / Pi	Notes
DHT22 (air temp / humidity)	DATA / VCC / GND	GPIO15 / 3.3 V / GND	10 k pull-up if the module has none
EC sensor (EC V2.1)	A0 / VCC / GND	GPIO34 / 3.3 V / GND	T1 unconnected; patched DFRobot_EC library
pH sensor (PH-4502C)	Po / V+ / G	GPIO35 via divider / 5 V / GND	10 k / 20 k divider is required
HC-SR04 (water level)	TRIG / ECHO / VCC / GND	GPIO32 / GPIO33 via divider / 5 V / GND	1 k / 2 k divider on ECHO; mounted above the tank, facing down
2-channel relay	IN1 / IN2	GPIO25 (Pump A) / GPIO26 (Pump B)	Active LOW; GND shared with the ESP32
1-channel relay	IN	GPIO27 (sampling pump)	Active LOW
ESP32 → Pi	USB	/dev/esp32 (udev rule)	115200 baud, one JSON line every 2 s
MaixCAM → Pi	USB-C	10.122.173.1 (camera), 10.122.173.100 (Pi)	Appears as a USB network adapter; the Pi gets its address automatically

Never put more than 3.3 V on an ESP32 pin. The pH module and the HC-SR04 run on 5 V, which is why their outputs go through voltage dividers. Keep the pH and EC probes in separate containers when testing; they interfere with each other.

3. How it works

3.1 Sensors (ESP32)

Every 2 seconds the ESP32 reads all sensors and prints one JSON line over USB:

```
{"temp":28.1,"hum":70.2,"ec":1.45,"ec_low":false,"ph":6.20,"level":9.8,"level_pct":72,
 "level_low":false,"ec_mv":237,"po_mv":2650,"dist_cm":10.2,
 "pump_a":true,"pump_b":true,"pump_auto":true,"pump_sample":true}
```

Field	Meaning
`temp`, `hum`	Air temperature (°C) and humidity (%) from the DHT22
`ec`	Electrical conductivity (mS/cm): how much nutrient is in the water
`ec_low`	true = below the ESP32's measuring floor (about 0.6 mS/cm), e.g. plain water
`ph`	pH value of the nutrient solution
`level`, `level_pct`	Water height (cm) and tank level (%) from the HC-SR04, after tank calibration
`level_low`	true = tank below 20 %
`pump_a`, `pump_b`, `pump_sample`	Relay state of each pump (true = running)
`pump_auto`	true = automatic EC/pH control, false = manual commands
`ec_mv`, `po_mv`, `dist_cm`	Raw sensor values, used for calibration and debugging

3.2 Pump control (ESP32)

The firmware keeps the nutrient solution inside the lettuce target: **EC 1.2–1.8 mS/cm** and **pH 5.5–6.5**.

- **Auto mode** (on at boot): Pump A and Pump B run together while EC or pH is outside the target, and stop when both are inside. They start again if either drifts out.
- **Sampling pump**: runs all the time to move water past the sensors (`sampleoff` / `sampleon`).
- **Tank-low protection**: when the tank is below 20 % (after tank calibration) **all three pumps are forced OFF**, in auto and manual, so they never run dry.
- **Manual**: `pumpaon`, `pumpaoff`, `pumpbon`, `pumpboff` switch to manual; `pumpauto` goes back to auto.
- A pH reading that is not a number stops Pumps A and B for safety.

3.3 Data logging (Raspberry Pi, bridge.py)

`bridge.py` reads the ESP32's serial port, publishes each reading to MQTT and writes it to a CSV file. Each time the ESP32 connects (plugged in, Pi reboot, bridge restart) a new file starts:

```
data/logs/log_0007_2026-09-23_07-28-38.csv

log_id,row,time,temp,hum,ec,ec_low,ph,level,level_pct,level_low,dist_cm,
ec_mv,po_mv,pump_a,pump_b,pump_auto,pump_sample
```

True/false values are stored as 1/0 and missing values as empty cells. The files open directly in Excel or LibreOffice Calc. The `data/` folder is not uploaded to GitHub.

3.4 Plant camera (MaixCAM + vision.py)

The MaixCAM is a small computer with its own camera and screen. Its `hydro_cam` app starts at boot, shows a live preview (to aim the camera) and hands a fresh photo to the Pi over the USB cable.

- **Every 10 s** `vision.py` takes a photo, checks it and publishes it to the dashboard.
- **Every 15 min** (or with **Save photo now**) it also saves the photo to `data/vision/<date>/` and adds a row to `vision_log.csv`. Photos older than 30 days are deleted, except one per day.
- It also writes `data/vision/live/latest.jpg` so the dashboard can still show the photo if the live MQTT link is blocked.

Health result	Rule (leaf colour check)
Healthy	Yellow/brown share of the leaf area is 8 % or less
Check leaves	Yellow/brown share above 8 %
Unhealthy	Yellow/brown share above 20 %
No plant in view	Less than 2 % of the photo is green (camera aim, focus or grow light)

This is a **colour rule, not an AI model**. A first growth-stage photo dataset now exists (section 3.6); the photos saved every 15 minutes can be added to it for size and growth-stage models.

3.5 Dashboard

A web page served by the Pi on port 8000. It connects to Mosquitto's websocket (port 9001) and updates live, without internet. It shows:

- Sensor tiles for pH, EC, water level, air temperature and humidity, each with its target band and an In range / Drifting / High / Low status.
- Pump cards for Nutrient A, Nutrient B and Sampling (ON/OFF and for how long), the pump mode (AUTO/MANUAL) and the tank state.
- The latest camera photo with its health result, plant coverage, yellow/brown share and greenness.
- Connection status for MQTT, the ESP32 and the camera. Old data is greyed out, never shown as if it were live.

3.6 Plant image dataset

`dataset/lettuce_growth_stages/` holds **41 lettuce photos sorted by growth stage**. It is the starting point for a camera AI model that tells the growth stage of the plants the MaixCAM sees.

Stage	Folder	Photos
Seedling	`seedling/`	18
Vegetative	`vegetative/`	6
Matured (ready to harvest)	`matured/`	17
Total		**41**

- The folder name is the label. `labels.csv` lists each photo with its stage, format, size and original file name.
- Mixed formats (35 JPEG, 4 AVIF, 2 WEBP) and sizes (413×550 to 2976×2232 px): convert and resize them (e.g. to 224×224 JPEG) before training.
- The classes are unbalanced (only 6 vegetative photos). Add more, including photos from `data/vision/`, before training a model.

4. What has been done

Area	Work	Status
ESP32 firmware	DHT22, EC and pH with calibration stored in EEPROM	**Done**
ESP32 firmware	HC-SR04 water level with tank calibration	**Done**
ESP32 firmware	Pump A/B auto control to EC 1.2–1.8 and pH 5.5–6.5; manual commands	**Done**
ESP32 firmware	Always-on sampling pump (GPIO27) and tank-low stop for all pumps	**Done**
Testing	Relay test sketch: both relays switched Pump A and Pump B correctly	**Done**
Raspberry Pi	Serial → MQTT bridge with auto-reconnect and fixed /dev/esp32 name	**Done**
Raspberry Pi	Mosquitto broker (local TCP + websocket for the dashboard)	**Done**
Data logging	CSV log per ESP32 connection, including pump columns	**Done**
Camera	MaixCAM app: auto-start, preview, photo server over USB	**Done**
Camera	vision.py: live photos, leaf colour health check, photo archive	**Done**
Camera AI	Growth-stage image dataset: 41 photos (seedling, vegetative, matured)	**Done**
Dashboard	Design plan and mockups (desktop overview, graphs, calibration)	**Done**
Dashboard	First live page: sensors, pumps, camera photo	**Done**
Dashboard	History charts, calibration wizards (full React dashboard)	**To do**
Camera AI	Size and growth-stage models (train on the dataset)	**To do**
Sensors	DS18B20 water temperature for EC compensation	**To do**
Safety	Password/ACL on MQTT; maximum run time for Pumps A/B	**To do**

Development history (GitHub commits)

Commit	What changed
6214b11	Initial commit: ESP32 firmware, serial bridge, README
5b9a0b8 · 97d15b4	Full README (wiring, calibration, troubleshooting); systemd, Mosquitto and udev configs
9de0bf0 · 27b1f4d	Sensor prototyping sketches, circuit diagram, relay test, HC-SR04 in the firmware
07c09f3 → 6a15904	Pump A/B relay control, tank-low stop, EC/pH target control, always-on sampling pump
bd111f6	CSV session logs with pump state
68434be	Dashboard design plan
2856219	MaixCAM plant camera with leaf colour health check
3272c0d · a066e70	First live dashboard page, with an HTTP fallback for the camera photo
02f4b24	This system guide (PDF)
24d1ede	Lettuce growth-stage image dataset (41 photos in 3 stages)

Current status (23 Sep 2026)

Part	State
ESP32 + bridge	**Running** · sending data with pump columns (log #0007)
Mosquitto	**Running**
MaixCAM	**Running** · photos reach the Pi; camera not yet aimed at the plants
vision.py / dashboard	**Started by hand** · install the two services (section 5.2) so they survive a reboot
pH probe	**Check** · reads 19.71, which means disconnected or not calibrated
Water level	**Check** · no echo from the HC-SR04 (level empty in the log)
EC	**Check** · `ec_low` (probe in plain water or dry)

5. How to run the system

5.1 First-time setup (already done on this Pi)

Upload `Full_code/Full_code.ino` to the ESP32 with the Arduino IDE (board: ESP32 Dev Module; libraries: DHT sensor library and the patched `DFRobot_EC` files in the same folder). Then on the Pi:

```
sudo apt install mosquitto mosquitto-clients python3-serial python3-paho-mqtt python3-pil
sudo cp ~/hydro/mosquitto/local.conf /etc/mosquitto/conf.d/
sudo cp ~/hydro/udev/99-esp32.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
sudo systemctl restart mosquitto
```

Install the camera app on the MaixCAM (one time):

```
scp -r ~/hydro/maixcam/hydro_cam root@10.122.173.1:/maixapp/apps/
ssh root@10.122.173.1 'echo hydro_cam > /maixapp/auto_start.txt && reboot'
ssh root@10.122.173.1 passwd          # change the factory password
```

5.2 Install the three services (start at every boot)

```
cd ~/hydro
sudo cp hydro-bridge.service hydro-vision.service hydro-dashboard.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now hydro-bridge hydro-vision hydro-dashboard
```

If `vision.py` or the dashboard are still running from a manual start, stop them first: `pkill -f vision.py; pkill -f 'http.server 8000'`

5.3 Daily use

- Power on the Pi with the ESP32 and the MaixCAM plugged in by USB. All services start by themselves.
- Open the dashboard: **http://localhost:8000** on the Pi, or **http://<pi-ip>:8000** from a phone on the same WiFi (find the IP with `hostname -I`).
- Check the three dots at the top right: MQTT, ESP32 and Camera should be green.
- Aim the camera at the lettuce using the preview on the MaixCAM screen. Take photos under the grow light.

5.4 Useful commands

Command	What it does
`systemctl status hydro-bridge hydro-vision hydro-dashboard`	Are the services running?
`journalctl -u hydro-bridge -f`	Follow the bridge log (ESP32 messages, new CSV files)
`journalctl -u hydro-vision -f`	Follow the camera log (saved photos, health results)
`mosquitto_sub -t 'hydro/#' -v`	Watch everything on MQTT (the image topic prints binary)
`mosquitto_pub -t hydro/cmd -m status`	Ask the ESP32 for its calibration and pump state
`mosquitto_pub -t hydro/cmd -m pumpauto`	Put the pumps back in automatic mode
`mosquitto_pub -t hydro/vision/cmd -m snap`	Save a camera photo now
`python3 ~/hydro/vision.py --once`	Test the camera: one photo, print the result
`ls -lt ~/hydro/data/logs | head`	Newest sensor CSV logs
`curl http://10.122.173.1:8080/status`	Is the camera app answering?
`sudo systemctl restart hydro-bridge`	Restart the bridge (starts a new CSV log)

6. Calibration

Send commands from the Arduino Serial Monitor (115200, Newline) or with `mosquitto_pub -t hydro/cmd -m <command>`. All values are stored in the ESP32 and survive reboots. Check them with `status`.

Sensor	Steps	Expected
EC	`enterec` → probe in 1413 µS/cm buffer → `calec` → probe in 12.88 mS/cm buffer → `calec` → `exitec`	~230 mV in 1413, ~2100 mV in 12.88; K between 0.5 and 1.5
pH	Probe in pH 7 buffer, wait 1–2 min → `ph7` → rinse → pH 4 buffer, wait → `ph4` (`phreset` = defaults)	Slope about 180 mV/pH
Tank level	Tank empty → `tankempty` → fill to MAX → `tankfull` (`tankreset` = defaults)	Empty and full at least 3 cm apart; sensor at least 2 cm above MAX

The tank-low pump stop only works after the tank is calibrated. After use, rinse the pH probe and put the cap back on with its storage solution; the glass bulb must never dry out.

Lettuce target ranges

Parameter	Target
EC	1.2–1.8 mS/cm (seedlings 0.5–1.0)
pH	5.5–6.5
Air temperature	18–26 °C (placeholder, confirm for your room)
Humidity	50–75 % (placeholder)

7. Troubleshooting

Problem	Fix
`/dev/esp32` missing	Check `lsusb`, re-plug the cable, reload the udev rules
No data on `hydro/sensors`	`journalctl -u hydro-bridge -f`; close the Arduino Serial Monitor (only one program can use the port)
pH stuck (e.g. 19.71)	Probe/BNC disconnected or not calibrated; check wiring, then calibrate
`ec_low: true`	Solution below about 0.6 mS/cm (normal for plain water)
Water level empty	HC-SR04 gets no echo: check wiring, the ECHO divider, and that it faces the water
Dashboard shows old data	Hard refresh (Ctrl+Shift+R). If the header says "Live link down", the browser can't reach MQTT on port 9001: restart the browser or turn off HTTPS-Only for the site. The camera photo still updates via HTTP
`hydro/vision/status` offline	`ping 10.122.173.1`; check the USB-C cable and that the MaixCAM shows the Hydro Cam preview
MaixCAM freezes / SSH hangs	Two programs used the camera. Reboot the MaixCAM. Never run camera scripts over SSH while its menu is running
Health says "No plant in view"	Aim the camera at the plants, check focus, switch on the grow light

8. Known issues and next steps

Safety items first

- **Pumps can dose without limit.** With a bad pH reading (such as 19.71) or a dry EC probe, the firmware sees "out of target" and runs Pumps A and B continuously. Add a maximum run time, or dose in short pulses (e.g. 2 s ON, wait 30–60 s, re-check). Until then, keep the nutrient pumps unplugged when the probes are not in solution.
- **MQTT has no password.** Anyone on the WiFi can send pump or reset commands to `hydro/cmd`. Add a Mosquitto password and an access list before using the dashboard from other devices.
- **MaixCAM factory password.** Change it with `passwd` (section 5.1).

Roadmap

- Full React dashboard: history charts from the CSV logs, pump activity lanes, calibration wizards (design in `docs/dashboard-design-plan.md`).
- FastAPI `api.py` for history and CSV download.
- Grow the growth-stage dataset in `dataset/lettuce_growth_stages/` (more vegetative photos, photos from the Hydro camera), then train size and growth-stage models (possibly on the MaixCAM's own NPU).
- DS18B20 water temperature for EC temperature compensation; grow-light relay; RTC battery for correct offline time.